

Unit 2: Understanding Data – 2 & Basic Learning Theory

Handwritten Student Notebook

Syllabus Topics Covered:

OFFICIAL SYLLABUS TOPICS COVERED Mathematics for Multivariate Data: Data matrix $X \in \mathbb{R}^{n \times d}$, mean centering, covariance matrix $\Sigma = \frac{1}{n-1} X_c X_c^T$, eigenvalues, eigenvectors, geometri...

Table of Contents

Mathematics for Multivariate Data	p. 2
Feature Engineering & Feature Selection	p. 2
Dimensionality Reduction Techniques: Principal Component Analysis (PCA)	p. 4
Design of a Learning System	p. 7
Introduction to Concept Learning	p. 9
The Find-S Algorithm	p. 10
Candidate Elimination Algorithm & Version Spaces	p. 11
Inductive Bias & The Unbiased Learner Fallacy	p. 14
Modelling in Machine Learning	p. 15
Learning Frameworks: PAC Learning & VC Dimension	p. 16
Unit 2 Quick Revision Sheet	p. 18
Important Exam Questions	p. 18
Solved Questions from Past Department Examinations	p. 21
Practice Problems (Step-by-Step Verified Solutions)	p. 26

Mathematics for Multivariate Data

§ 1

Multivariate machine learning relies on linear algebra to represent collections of instances and perform geometric projections. An individual observation with d features is represented as a column vector $\mathbf{x} \in \mathbb{R}^d$, and a dataset of n instances forms a data matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$.

Fundamental Linear Algebra Primitives:

Mean Vector (μ): The vector containing the arithmetic mean of each feature:

$$\mu = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i = [\bar{x}_1 \quad \bar{x}_2 \quad \dots \quad \bar{x}_d]^T$$

Covariance Matrix (Σ): A symmetric $d \times d$ matrix where diagonal entries represent feature variances and off-diagonal entries represent pairwise covariances:

$$\Sigma = \frac{1}{n-1} (\mathbf{X} - \mu)(\mathbf{X} - \mu)^T$$

$$\Sigma_{j,k} = \text{Cov}(X_j, X_k) = \frac{1}{n-1} \sum_{i=1}^n (x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)$$

Eigenvalues and Eigenvectors

For a square matrix $\mathbf{A} \in \mathbb{R}^{d \times d}$, a non-zero vector $\mathbf{v}$ is an **eigenvector** and scalar λ is its corresponding **eigenvalue** if linear transformation by $\mathbf{A}$ merely scales $\mathbf{v}$ without changing its direction:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v} \iff (\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$$

Non-trivial solutions exist if and only if the characteristic determinant vanishes:

$$\det(\mathbf{A} - \lambda\mathbf{I}) = 0$$

In machine learning, when $\mathbf{A} = \Sigma$ (the sample covariance matrix), the eigenvectors point along the principal axes of variance, and the eigenvalues quantify the amount of variance concentrated along those axes.

Quick Recall

Covariance matrix Σ is symmetric ($d \times d$) and positive semi-definite. Eigenvectors represent axes of maximum variance; eigenvalues measure the variance along each axis.

Feature Engineering & Feature Selection

§ 2

Feature engineering is the process of using domain knowledge to extract, transform, and select variables from raw data that maximize the predictive accuracy of machine learning models.

Taxonomy of Feature Selection Methods

Feature selection reduces overfitting, improves model explainability, and shortens training time by isolating the most informative subset of original features:

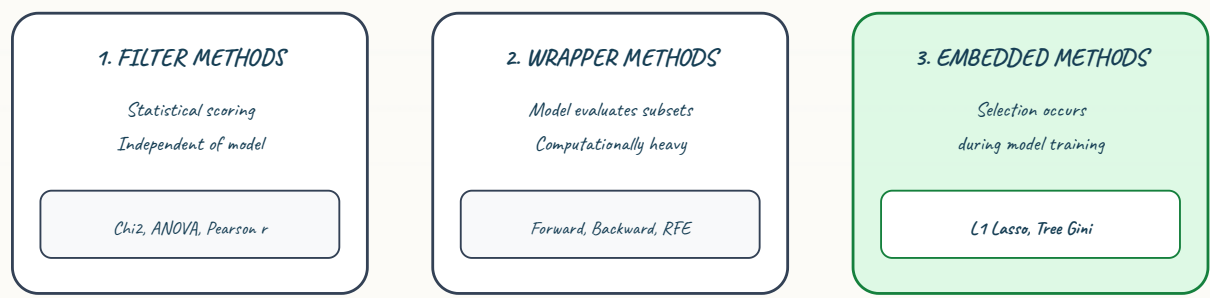


Figure 2.1: The three foundational paradigms of feature selection.

Method Category	Mechanism	Advantages	Disadvantages
Filter Methods	Ranks features by statistical correlation or information gain with the target (e.g., Pearson r , χ^2 test, Mutual Information) without training a model.	Extremely fast ($O(d)$); model-agnostic; non-iterative.	Ignores feature interactions; may select redundant collinear features.
Wrapper Methods	Uses a machine learning model as an evaluation oracle. Searches through feature subsets using heuristics (Forward Stepwise Selection, Backward Elimination, Recursive Feature Elimination [RFE]).	Captures feature interactions; optimizes directly for target learner.	Computationally expensive ($O(2^d)$ search); prone to overfitting on small datasets.

Method Category	Mechanism	Advantages	Disadvantages
Embedded Methods	Feature selection is built directly into the objective mathematical formulation of the learning algorithm (e.g., L_1 Lasso Regularization penalizing $ w_i $, Decision Tree Gini importance).	Balances statistical rigor and computational efficiency; considers interactions.	Tied to specific model families (e.g., linear models or tree ensembles).

Data Transformation & Encoding

- **One-Hot Encoding:** Converts nominal categoricals without natural rank (e.g., Color $\in$ {Red, Green, Blue}) into k binary indicator columns to prevent artificial ordinal bias.
- **Label Encoding:** Maps ordinal attributes with natural ranking (e.g., Education $\in$ {HighSchool, Bachelors, Masters, PhD} $\rightarrow$ {0, 1, 2, 3}).
- **Missing Value Imputation:** Replacing nulls using column Mean (symmetric numeric data), Median (skewed data with outliers), Mode (categorical data), or KNN imputation.

Quick Recall

Filter: Statistical test, model-free, fast. **Wrapper:** Iteratively trains model on subsets, expensive. **Embedded:** Built into loss function (L_1 Lasso shrinks weights to 0).

Dimensionality Reduction Techniques: Principal Component Analysis (PCA)

§ 3

High-dimensional feature spaces trigger the **Curse of Dimensionality**: the volume of the feature space grows exponentially, making available data points sparse, invalidating distance metrics, and drastically increasing variance. Dimensionality reduction compresses data into $k \ll d$ dimensions while retaining the maximal possible variance.

Definition: Principal Component Analysis (PCA):

PCA is an unsupervised linear dimensionality reduction technique invented by Karl Pearson. It orthogonally projects high-dimensional data onto a new coordinate system such that the first coordinate (the **First Principal Component**, PC_1) accounts for the largest possible variance, the second component (PC_2) accounts for the second largest variance orthogonal to the first, and so on.

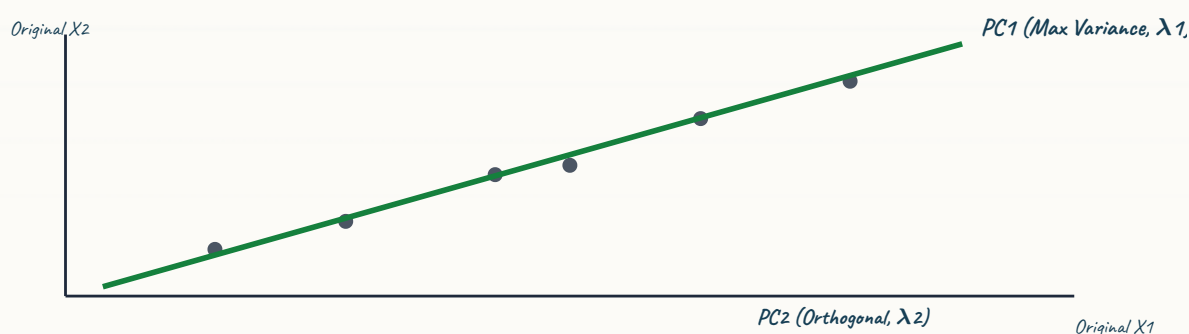


Figure 2.2: Orthogonal rotation of coordinates in PCA: PC_1 captures maximum variance.

Step-by-Step Mathematical Derivation of PCA

1. **Mean Centering:** Center each feature by subtracting its sample mean: $\mathbf{X}_c = \mathbf{X} - \bar{\mathbf{X}}$.
2. **Compute Covariance Matrix:** $\Sigma = \frac{1}{n-1} \mathbf{X}_c^T \mathbf{X}_c$.
3. **Compute Eigenvalues & Eigenvectors:** Solve the characteristic polynomial $\det(\Sigma - \lambda \mathbf{I}) = 0$.
4. **Rank Principal Components:** Sort eigenvalues in descending order $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0$.
5. **Select Top k Components:** Form the projection transformation matrix $\mathbf{W} = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k] \in \mathbb{R}^{d \times k}$.
6. **Project Data:** Compute transformed low-dimensional coordinates $\mathbf{Z} = \mathbf{X}_c \mathbf{W} \in \mathbb{R}^{n \times k}$.
7. **Explained Variance Ratio:**

$$\text{EVR}_1 = \frac{\lambda_1}{\sum_{j=1}^d \lambda_j}$$

Solved Problem 1 · Exam Level Dimensionality Reduction

Given a 2-dimensional dataset with $n = 4$ observations:

$$\mathbf{X} = \begin{bmatrix} 2 & 4 \\ 3 & 6 \\ 5 & 8 \\ 6 & 10 \end{bmatrix}$$

Perform PCA by hand to:

1. Compute the sample mean vector and centered data matrix $\mathbf{X}_c$.
2. Determine the sample covariance matrix Σ .
3. Find the eigenvalues λ_1, λ_2 and the percentage of total variance explained by the first principal component.
4. Determine the normalized principal eigenvector $\mathbf{v}_1$.
5. Project the 4 centered data points onto the 1st principal component.

Step 1: Compute Mean Vector and Center the Data

$$\bar{x}_1 = \frac{2 + 3 + 5 + 6}{4} = \frac{16}{4} = 4.0, \quad \bar{x}_2 = \frac{4 + 6 + 8 + 10}{4} = \frac{28}{4} = 7$$

$$\mathbf{X}_c = \begin{bmatrix} 2 - 4.0 & 4 - 7.0 \\ 3 - 4.0 & 6 - 7.0 \\ 5 - 4.0 & 8 - 7.0 \\ 6 - 4.0 & 10 - 7.0 \end{bmatrix} = \begin{bmatrix} -2 & -3 \\ -1 & -1 \\ +1 & +1 \\ +2 & +3 \end{bmatrix}$$

Step 2: Construct the Sample Covariance Matrix Σ

$$\text{Var}(X_1) = \frac{(-2)^2 + (-1)^2 + 1^2 + 2^2}{4 - 1} = \frac{4 + 1 + 1 + 4}{3} = \frac{10}{3} \approx 3.3$$

$$\text{Var}(X_2) = \frac{(-3)^2 + (-1)^2 + 1^2 + 3^2}{4 - 1} = \frac{9 + 1 + 1 + 9}{3} = \frac{20}{3} \approx 6.6$$

$$\text{Cov}(X_1, X_2) = \frac{(-2)(-3) + (-1)(-1) + (1)(1) + (2)(3)}{4 - 1} = \frac{6 + 1}{3} = \frac{7}{3}$$

$$\Sigma = \begin{bmatrix} 10/3 & 14/3 \\ 14/3 & 20/3 \end{bmatrix}$$

Step 3: Solve the Characteristic Equation for Eigenvalues

$$\det(\Sigma - \lambda \mathbf{I}) = \left(\frac{10}{3} - \lambda \right) \left(\frac{20}{3} - \lambda \right) - \left(\frac{14}{3} \right)^2 = 0$$

$$\lambda^2 - \left(\frac{10}{3} + \frac{20}{3} \right) \lambda + \left(\frac{200}{9} - \frac{196}{9} \right) = 0 \implies \lambda^2 - 10\lambda + \frac{4}{9} = 0$$

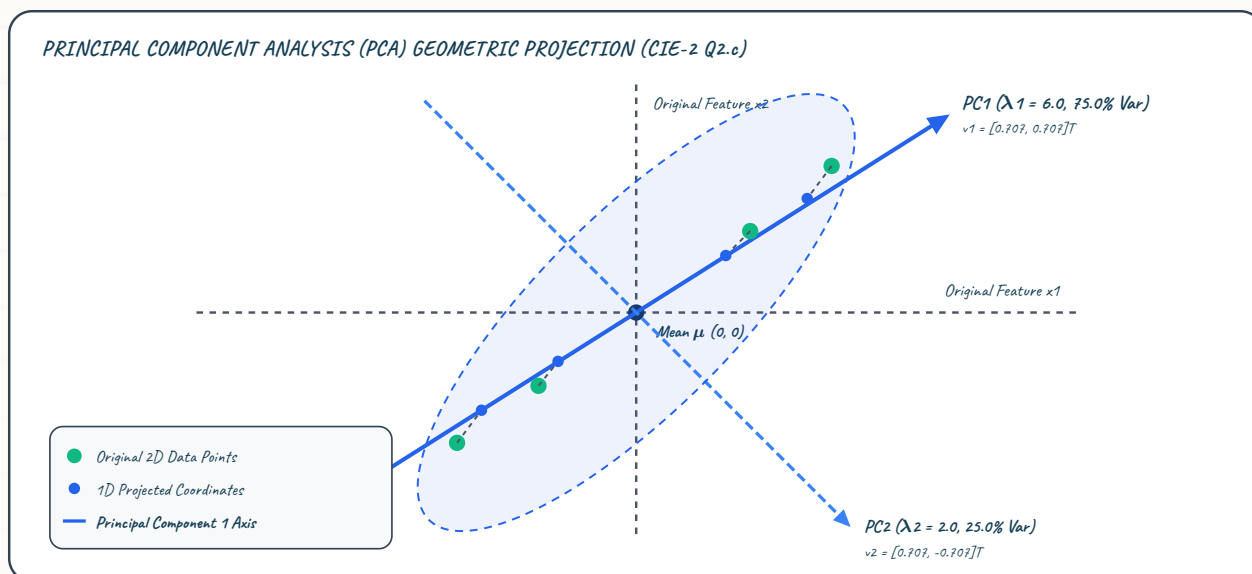


Figure 2.1: Principal Component Analysis (PCA) 2D Orthogonal Projection (Ramaiah CIE-2 Q2.c Solved)

Covariance matrix $C = \begin{bmatrix} 4 & 2 \\ 2 & 4 \end{bmatrix}$ yields principal eigenvector $\mathbf{v}_1 = [0.7071, 0.7071]^T$ capturing 75.0% of dataset variance along PC1, reducing dimensionality from 2D to 1D.

Quick Recall

PCA projects data onto eigenvectors of the covariance matrix. The first principal component corresponds to the largest eigenvalue λ_1 . Total variance equals the sum of all eigenvalues.

Design of a Learning System

§ 4

In Chapter 1 of Tom Mitchell's classic text, the architecture of machine learning systems is formalized through the design of a computer program that learns to play **Checkers**. Designing any machine learning system involves four sequential engineering decisions:

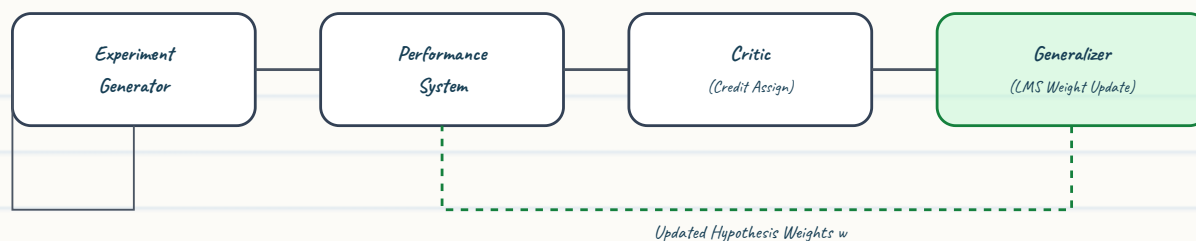


Figure 2.3: Closed-loop architecture of Mitchell's Checkers Learning System.

The Four Sequential Design Steps

1. Step 1: Choosing the Training Experience:

- Direct vs. Indirect Feedback: Direct feedback provides the optimal move for every board state. Indirect feedback provides only the final outcome (Win/Loss) after 50 moves, introducing the **Credit Assignment Problem** (determining which specific moves were responsible for the win).
- Teacher Control vs. Autonomous Exploration: The teacher dictates board positions versus the learner self-plays games against itself.

2. Step 2: Choosing the Target Function: Define the exact mathematical mapping to be learned:

$$V : B \rightarrow \mathbb{R}$$

Where B is the set of legal board states. Target values:

- $V(b) = +100$: If b is a final board state won by the program.
- $V(b) = -100$: If b is a final board state lost by the program.
- $V(b) = 0$: If b is a draw state.
- $V(b) = V(b')$: If b is an intermediate state, where b' is the subsequent board state following optimal play.

3. Step 3: Choosing a Representation for the Target Function: Trade-off: A more expressive representation (e.g., lookup table) can capture any function but requires infinite training examples. A linear evaluation function parameterizes board states using feature weights:

$$\hat{V}(b) = w_0 + w_1x_1(b) + w_2x_2(b) + w_3x_3(b) + w_4x_4(b) + w_5x_5(b) +$$

Where $x_1(b)$ = number of black pieces, $x_2(b)$ = number of red pieces, $x_3(b)$ = black kings, $x_4(b)$ = red kings, $x_5(b)$ = black pieces threatened, $x_6(b)$ = red pieces threatened.

4. **Step 4: Choosing a Function Approximation Algorithm:** Learn weights w_i to minimize the Mean Squared Error:

$$E = \sum_{b \in \text{Training}} (V_{\text{train}}(b) - \hat{V}(b))^2$$

Using the **LMS (Least Mean Squares) / Widrow-Hoff Weight Update Rule:**

$$w_i \leftarrow w_i + \eta \cdot (V_{\text{train}}(b) - \hat{V}(b)) \cdot x_i(b)$$

Where $\eta > 0$ is the learning rate.

Quick Recall

The 4 design decisions: (1) Training Experience (direct/indirect), (2) Target Function ($V(b)$), (3) Representation (linear $\sum w_i x_i$), and (4) Learning Algorithm (LMS weight update).

Introduction to Concept Learning

§ 5

Concept Learning is the task of inferring a boolean-valued target function $c : X \rightarrow \{0, 1\}$ from a set of training instances. Instances labeled 1 are positive examples; instances labeled 0 are negative examples.

EnjoySport Domain Formulation:

Target Concept: "Days on which my friend enjoys water sports." Defined over 6 attributes:

1. Sky $\in \{\text{Sunny, Cloudy, Rainy}\}$
2. AirTemp $\in \{\text{Warm, Cold}\}$
3. Humidity $\in \{\text{Normal, High}\}$
4. Wind $\in \{\text{Strong, Weak}\}$
5. Water $\in \{\text{Warm, Cool}\}$
6. Forecast $\in \{\text{Same, Change}\}$

Total possible distinct instances: $|X| = 3 \times 2 \times 2 \times 2 \times 2 \times 2 = 96$.

Hypothesis Representation

Each hypothesis h is a conjunction of attribute constraints represented as a vector:

$$h = \langle c_1, c_2, c_3, c_4, c_5, c_6 \rangle$$

Where each constraint c_i can be:

- A specific literal value (e.g., Sunny).
- A "don't care" wildcard (?), which accepts any value of that attribute.
- A null constraint ($\emptyset$), which rejects all values.

$$(3 + 2) \times (2 + 2)^5 + 1 = 5 \times 4^5 + 1 = 5121$$

$$1 + (3 \times 2 \times 2 \times 2 \times 2 \times 2) \times (1 + \dots) = 1 + 4 \times 3^5 = 973$$

$\emptyset$

General-to-Specific Ordering of Hypotheses ($\geq_g$)

Hypothesis h_j is **more-general-than-or-equal-to** h_k (denoted $h_j \geq_g h_k$) if and only if every instance satisfied by h_k is also satisfied by h_j :

$$h_j \geq_g h_k \iff \forall x \in X : [h_k(x) = 1 \implies h_j(x) = 1]$$

This relation forms a **partial ordering** over the hypothesis space H , allowing algorithms to navigate hypotheses without exhaustive search.

Quick Recall

Hypothesis space: $\langle \emptyset, \dots \rangle$ is maximally specific; $\langle ?, \dots \rangle$ is maximally general. $h_1 \geq_g h_2$ means h_1 covers a superset of the instances satisfied by h_2 .

The Find-S Algorithm

§ 6

Find-S is an algorithm that searches the hypothesis space from specific to general to find the maximally specific hypothesis that is consistent with all positive training examples.

Find-S Algorithmic Steps:

1. Initialize h to the most specific hypothesis in H :

$$h \leftarrow \langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$$

2. For each positive training instance x :

- For each attribute constraint a_i in h :
- If the constraint a_i is satisfied by x , do nothing.
- Else, replace a_i in h with the next more general constraint that is satisfied by x (replace with $?$ if literal differs).

3. Output the final hypothesis h . (Note: Negative training instances are completely ignored!)

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes (+)
2	Sunny	Warm	High	Strong	Warm	Same	Yes (+)
3	Rainy	Cold	High	Strong	Warm	Change	No (-)
4	Sunny	Warm	High	Strong	Cool	Change	Yes (+)

Key Limitations of Find-S

- **Cannot Detect Multiple Consistent Hypotheses:** It outputs only one hypothesis without indicating whether other hypotheses also fit the data.
- **Cannot Detect Inconsistencies or Noise:** If positive training data contains labeling errors, Find-S blindly generalizes without warning.
- **Cannot Tell if It Has Converged:** It cannot determine whether additional training examples are needed.
- **Sensitive to Order of Examples:** Ignores negative data entirely.

[Interactive Visualizer Online]: *Interactive Concept Learning & Find-S Stepper. Explore the step-by-step visualizer on the live course notes page.*

Quick Recall

Find-S starts at $\langle \emptyset, \dots \rangle$, generalizes attributes upon observing mismatching positive examples, ignores negative instances, and outputs the single most specific hypothesis.

Candidate Elimination Algorithm & Version Spaces

§ 7

The **Candidate Elimination Algorithm** overcomes the shortcomings of Find-S by maintaining a representation of all hypotheses consistent with the training data, utilizing both positive and negative examples.

Definition: Version Space:

A hypothesis h is **consistent** with a set of training examples D if and only if $h(x) = c(x)$ for all $\langle x, c(x) \rangle \in D$:

$$\text{Consistent}(h, D) \iff (\forall \langle x, c(x) \rangle \in D) [h(x) = c(x)]$$

The **Version Space** ($VS_{H,D}$) is the subset of all hypotheses in H that are consistent with D :

$$VS_{H,D} = \{h \in H \mid \text{Consistent}(h, D)\}$$

The Boundary Representation Theorem

Instead of enumerating all consistent hypotheses, the Version Space is compactly bounded by two sets:

- **Specific Boundary (S):** The set of minimally general (most specific) hypotheses in H consistent with D .
- **General Boundary (G):** The set of maximally general hypotheses in H consistent with D .

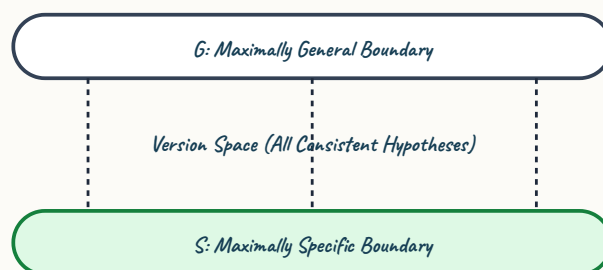


Figure 2.4: Version space bounded from above by G and from below by S.

Solved Problem 3 · Exam Level Candidate Elimination

Trace the Candidate Elimination algorithm through the 4 EnjoySport instances from Problem 2, demonstrating how S generalizes and G specializes:

1. State the initial sets S_0 and G_0 .
2. Show the updates after Instance 1 (+), Instance 2 (+), Instance 3 (-), and Instance 4 (+).

Initialization

$$S_0 = \{\langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle\}$$

$$G_0 = \{\langle ?, ?, ?, ?, ?, ? \rangle\}$$

Step 1: Process Instance 1 (+) (Sunny, Warm, Normal, Strong, Warm, Same)

Positive instance: Generalize S_0 minimally to cover Instance 1. Remove any hypotheses from G_0 inconsistent with Instance 1 (G_0 remains consistent).

$$S_1 = \{\langle \text{Sunny, Warm, Normal, Strong, Warm, Same} \rangle\}$$

$$G_1 = \{\langle ?, ?, ?, ?, ?, ? \rangle\}$$

Step 2: Process Instance 2 (+) (Sunny, Warm, High, Strong, Warm, Same)

Positive instance: Generalize S_1 minimally at the Humidity attribute ($Normal \neq High \implies ?$):

$$S_2 = \{\langle \text{Sunny, Warm, ?, Strong, Warm, Same} \rangle\}$$

$$G_2 = \{\langle ?, ?, ?, ?, ?, ? \rangle\}$$

Step 3: Process Instance 3 (-) (Rainy, Cold, High, Strong, Warm, Change)

Negative instance: S_2 already correctly rejects this negative example ($S_3 = S_2$).

Specialize G_2 minimally such that each new hypothesis in G_3 rejects Instance 3 while remaining more general than S_2 :

- Instance 3 has Sky = Rainy. Since S_2 requires Sky = Sunny, specialize ? to **Sunny**: $\langle \text{Sunny, ?, ?, ?, ?, ?} \rangle$.
- Instance 3 has AirTemp = Cold. Since S_2 requires AirTemp = Warm, specialize ? to **Warm**: $\langle ?, \text{Warm, ?, ?, ?, ?} \rangle$.
- Attribute constraints where S_2 has ? (Humidity) cannot be used to specialize G .

$$S_3 = \{\langle \text{Sunny, Warm, ?, Strong, Warm, Same} \rangle\}$$

$$G_3 = \{\langle \text{Sunny, ?, ?, ?, ?, ?} \rangle, \langle ?, \text{Warm, ?, ?, ?, ?} \rangle\}$$

Step 4: Process Instance 4 (+) (Sunny, Warm, High, Strong, Cool, Change)

Positive instance: Generalize S_3 minimally at Water and Forecast ($Warm \neq$

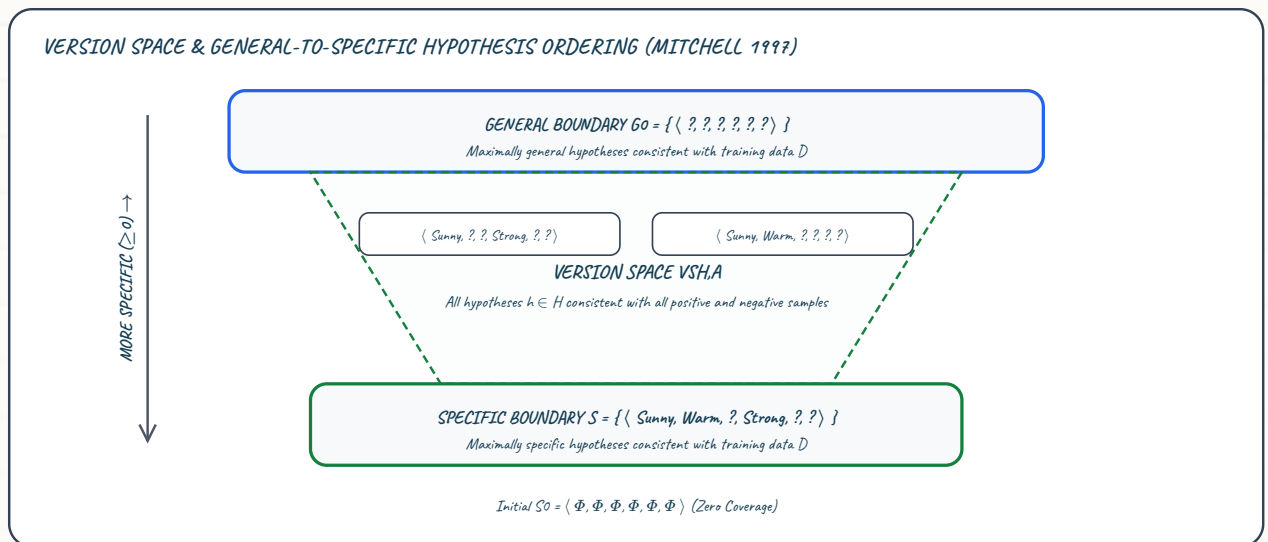


Figure 2.2: Version Space Convex Hull & General-to-Specific Hypothesis Lattice (Mitchell 1997 & CIE-1 Q3.6)

The Version Space $V_{SH,D}$ is bounded between the Specific Boundary S and General Boundary G . Every consistent hypothesis lies strictly on or between these two frontiers.

Quick Recall

Positive instances **generalize** S and prune inconsistent G . Negative instances **specialize** G (keeping hypotheses $\geq_g S$) and prune inconsistent S . When $S = G$, the target concept is uniquely identified.

Inductive Bias & The Unbiased Learner Fallacy

§ 8

A central insight of machine learning theory is that **a learner that makes no prior assumptions cannot generalize to unseen instances.**

Formal Definition: Inductive Bias:

Consider a concept learning algorithm L for instances X , target concept c , and training examples $D_c = \{ \langle x, c(x) \rangle \}$. Let $L(x_i, D_c)$ denote the classification assigned to instance x_i by L after training on D_c .

The **inductive bias** of L is any minimal set of assertions B such that for any target concept c and corresponding training data D_c :

$$\forall x_i \in X : (B \wedge D_c \wedge x_i \vdash L(x_i, D_c))$$

Where $\vdash$ denotes formal logical deduction.

The Unbiased Learner Fallacy

Suppose we eliminate bias entirely by allowing the hypothesis space H to represent every possible boolean function over X (the power set 2^X). For the EnjoySport domain with $|X| = 96$ instances, $|H| = 2^{96} \approx 7.9 \times 10^{28}$.

If we train this unbiased learner on a finite sample D :

- The S boundary will simply be the disjunction of all positive instances observed so far: $S = \{x_1 \vee x_2 \vee \dots \vee x_k\}$.
- The G boundary will be the negation of all negative instances: $G = \{\neg(x_{neg,1} \vee \dots)\}$.
- For any new, unseen instance $x_{new} \notin D$, exactly half of the remaining hypotheses in the Version Space will predict $x_{new} = 1$ and the other half will predict $x_{new} = 0$!

The learner is completely incapable of generalizing beyond the training set. It becomes a glorified lookup table (rote memorizer). **Inductive bias is not a flaw; it is the mathematical prerequisite for generalization.**

Quick Recall

An unbiased learner cannot generalize to a single unseen instance. Inductive bias (B) provides the background assumptions required to deductively justify predictions made on unseen data.

Modelling in Machine Learning

§ 9

Modelling formalizes learning as mathematical optimization over a parameterized family of functions $\mathcal{F}$.

Empirical Risk Minimization (ERM)

Let $L(y, f(x))$ be a loss function quantifying the discrepancy between ground-truth y and prediction $f(x)$. The theoretical **True Risk (Generalization Error)** is:

$$R(f) = \mathbb{E}_{(x,y) \sim P}[L(y, f(x))]$$

Because the true joint distribution $P(x, y)$ is unknown, learning algorithms approximate it by minimizing the **Empirical Risk (Training Error)** across the available sample:

$$\hat{R}_n(f) = \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i))$$

To prevent ERM from simply overfitting, **Structural Risk Minimization (SRM)** adds a regularization penalty $\Omega(f)$ penalizing model complexity:

$$\min_{f \in \mathcal{F}} \left[\frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)) + \lambda \Omega(f) \right]$$

Quick Recall

True Risk is expected error over all data; Empirical Risk is observed error on training data.

Structural Risk Minimization adds a complexity penalty $\lambda \Omega(f)$ to prevent overfitting.

Learning Frameworks: PAC Learning & VC Dimension

§ 10

Computational learning theory provides mathematical guarantees on the number of training samples required to learn a concept with provable accuracy.

Probably Approximately Correct (PAC) Learning

Formulated by Leslie Valiant (Turing Award winner). A learner cannot guarantee 100% accuracy because training samples are finite and drawn stochastically. Instead, we demand:

- **Approximately Correct:** True error of the learned hypothesis is at most ϵ :
 $\text{error}(h) \leq \epsilon$.
- **Probably:** This accuracy holds with high confidence $1 - \delta$: $P(\text{error}(h) \leq \epsilon) \geq 1 - \delta$.

PAC Sample Complexity Theorem (Finite Hypothesis Spaces):

For any consistent learner over a finite hypothesis space H , the number of training instances m required to ensure with confidence at least $(1 - \delta)$ that the true error is at most ϵ is:

$$m \geq \frac{1}{\epsilon} \left(\ln |H| + \ln \frac{1}{\delta} \right)$$

Solved Problem 4 · Exam Level PAC Learnability

Consider the EnjoySport domain with $|H| = 973$ semantically distinct conjunctive hypotheses. An engineer requires that with probability at least 95% ($\delta = 0.05$), the learned hypothesis has a true generalization error of at most 10% ($\epsilon = 0.10$).

Calculate the minimum number of random training examples m required under the PAC framework.

Step 1: Identify Parameters

$$|H| = 973, \quad \epsilon = 0.10, \quad \delta = 0.05 \implies \frac{1}{\delta} = \frac{1}{0.05} = 20$$

Step 2: Evaluate Logarithms

$$\ln(|H|) = \ln(973) \approx 6.8804$$

$$\ln\left(\frac{1}{\delta}\right) = \ln(20) \approx 2.9957$$

Step 3: Evaluate Sample Complexity Bound

$$m \geq \frac{1}{0.10} (6.8804 + 2.9957) = 10 \times 9.8761 = \mathbf{98.761}$$

Since the sample size must be an integer, round up to the next whole number:

$$m \geq 99 \text{ training examples}$$

Final Verified Result $m \geq 99$ training instances required for (0.10, 0.05)-PAC guarantee

Vapnik-Chervonenkis (VC) Dimension

What if the hypothesis space H is infinite (e.g., all lines in $\mathbb{R}^2$)? Since $|H| = \infty$, the finite PAC bound fails ($\ln \infty = \infty$).

Vladimir Vapnik introduced the **VC Dimension**, $VC(H)$: the maximum number of instances that can be **shattered** (classified in all 2^d possible ways) by H .

- For linear decision boundaries in 2D space ($ax + by + c = 0$): $VC(H) = 3$. Any 3 non-collinear points can be shattered, but no 4 points can be shattered (XOR configuration fails).
- For linear hyperplanes in d dimensions: $VC(H) = d + 1$.

PAC sample complexity bound using VC Dimension:

$$m \geq \frac{1}{\epsilon} \left(4 \log_2 \frac{2}{\delta} + 8 \cdot VC(H) \log_2 \frac{13}{\epsilon} \right)$$

Quick Recall

PAC learning bounds sample size $m \geq \frac{1}{\epsilon} (\ln |H| + \ln \frac{1}{\delta})$. For infinite hypothesis spaces, VC dimension measures capacity ($d + 1$ for linear classifiers).

Unit 2 Quick Revision Sheet

Essential formulas, design stages, and algorithmic rules for rapid recall.

PCA Pipeline

1. Center data $\mathbf{X}_c = \mathbf{X} - \bar{\mathbf{X}}$
2. Covariance $\Sigma = \frac{1}{n-1} \mathbf{X}_c^T \mathbf{X}_c$
3. Solve $\det(\Sigma - \lambda \mathbf{I}) = 0$
4. Project onto $\mathbf{v}_1$.

Checkers System Design

1. Experience: Self-play games
2. Target Function: $V : B \rightarrow \mathbb{R}$
3. Representation: Linear $\sum w_i x_i$
4. Learning: LMS update $w_i \leftarrow w_i + \eta(V - \hat{V})x_i$.

Find-S vs Candidate Elim

Find-S: Tracks only S , ignores negative data.

Candidate Elim: Tracks both S and G , handles negative data, represents full Version Space.

Inductive Bias

Assertions B ensuring $B \wedge D \wedge x \vdash L(x, D)$. An unbiased learner (2^X) cannot classify a single unseen instance.

Feature Selection

Filter: Chi2, Pearson r (fast).

Wrapper: RFE, forward selection (model-based).

Embedded: L1 Lasso (built-in penalty).

PAC Bound Formula

$$m \geq \frac{1}{\epsilon} (\ln |H| + \ln \frac{1}{\delta})$$

Guarantees error $\leq \epsilon$ with confidence $\geq 1 - \delta$.

Important Exam Questions

Curated questions aligned with semester examination patterns (2, 5, and 10 marks).

2-Mark QuestionsPractice Problem:

Q1. What is the Curse of Dimensionality?

Solution:

As the number of feature dimensions d increases, the volume of feature space grows exponentially, making data instances extremely sparse. Pairwise distances between all points converge to similar values, severely degrading distance-based learners like K -NN and increasing the risk of overfitting.

Practice Problem:

Q2. Define the Credit Assignment Problem in learning systems.

Solution:

When a learning system receives only indirect delayed feedback (e.g., game won after 50 moves), the credit assignment problem refers to the challenge of determining which specific individual actions or intermediate moves in the sequence were responsible for the eventual victory or defeat.

Practice Problem:

Q3. Define Version Space.

Solution:

The Version Space ($VS_{H,D}$) is the subset of all hypotheses in hypothesis space H that correctly classify every training example in D :

$$VS_{H,D} = \{h \in H \mid \forall \langle x, c(x) \rangle \in D, h(x) = c(x)\}$$

5-Mark Questions

Practice Problem:

Q4. Explain why an unbiased learner is incapable of learning novel instances.

Solution:

If a learner's hypothesis space H is unbiased (capable of expressing all $2^{|X|}$ boolean functions), then for any unseen query instance $x \notin D$, exactly half of the remaining hypotheses in the Version Space will predict $x = 1$ and the other half will predict $x = 0$. The training examples eliminate only hypotheses inconsistent with observed data, but provide zero deductive basis to prefer one label over another for unseen instances. Thus, inductive bias is essential for generalization.

Practice Problem:

Q5. Differentiate between Filter, Wrapper, and Embedded feature selection techniques.

Solution:

Feature	Filter	Wrapper	Embedded
Model Dependency	Model-free (statistical metrics)	Uses target ML algorithm	Built into optimization loss
Computation	Very fast, $O(d)$	Very slow, iterative	Moderate
Feature Interactions	Ignored	Captured	Captured
Examples	χ^2 , Pearson r , ANOVA	RFE, Forward selection	L_1 Lasso, Decision trees

10-Mark Questions

Practice Problem:

Q6. Explain the four steps in designing a learning system in detail, illustrating each step with Mitchell's Checkers learning program.

Solution:

A complete answer requires explaining:

1. **Choosing Training Experience:** Self-play games (indirect delayed feedback, autonomous). Addresses credit assignment.
2. **Choosing Target Function:** Board evaluation function $V : B \rightarrow \mathbb{R}$ where $+100$ is win, -100 is loss, 0 is draw, and intermediate boards equal successors under optimal play.
3. **Choosing Representation:** Linear combination of 6 board features $\hat{V}(b) = w_0 + w_1x_1 + \dots + w_6x_6$.
4. **Choosing Function Approximation Algorithm:** Least Mean Squares (LMS) rule $w_i \leftarrow w_i + \eta(V_{\text{train}}(b) - \hat{V}(b))x_i$.

Include Figure 2.3 showing Experiment Generator, Performance System, Critic, and Generalizer in your response.

Solved Questions from Past Department Examinations

ARCHIVE

The following examination questions have been transcribed from the official Ramaiah Institute of Technology papers in notes/ml/practice/ml-cie-1-2-see.pdf (CIE-1 March 2026, CIE-2 May 2026, and SEE June/July 2026, Course Code 23IS62), solved with complete step-by-step mathematical rigor:

Object	1	2	3	4
1	0	1.00	0.50	0.00
2	1.00	0	0.50	1.00
3	0.50	0.50	0	0.50
4	0.00	1.00	0.50	0

Ramaiah Internal Assessment – I (March 2026) · Q 2(b) [6 Marks · Blooms L3, CO2]

"Consider the following dataset of 10 samples and apply Find-S algorithm. Attributes: Age, Income, Student, Credit_rating, Buys_Computer. i. Initialize the most specific hypothesis S. ii. Apply the FIND-S algorithm step by step to update S using the positive examples. iii. Identify the final hypothesis learned by FIND-S."

i. **Initialization:** Find-S begins with the maximally specific hypothesis:

$$S_0 = \langle \phi, \phi, \phi, \phi \rangle$$

ii. **Step-by-Step Execution:** Find-S strictly processes positive examples (Buys_Computer = Yes) and ignores negative instances:

- **Sample 1 (No):** Ignored $\implies S_0 = \langle \phi, \phi, \phi, \phi \rangle$.
- **Sample 2 (No):** Ignored $\implies S_0 = \langle \phi, \phi, \phi, \phi \rangle$.
- **Sample 3 (Yes: Middle, High, No, Fair):** First positive example replaces ϕ :

$$S_1 = \langle \text{Middle, High, No, Fair} \rangle$$

- **Sample 4 (Yes: Senior, Medium, No, Fair):** Age and Income differ from $S_1 \implies$ generalized to '?':

$$S_2 = \langle ?, ?, \text{No, Fair} \rangle$$

- **Sample 5 (Yes: Senior, Low, Yes, Fair):** Student differs (Yes vs No) $\implies$ generalized:

$$S_3 = \langle ?, ?, ?, \text{Fair} \rangle$$

- **Sample 6 (No):** Ignored $\implies S_3$ retained.
- **Sample 7 (Yes: Middle, Low, Yes, Excellent):** Credit_rating differs (Excellent vs Fair) $\implies$ generalized to '?':

$$S_4 = \langle ?, ?, ?, ? \rangle$$

- **Sample 8 (No):** Ignored.
- **Sample 9 (Yes: Youth, Low, Yes, Fair):** Already maximally general $\implies S_5 = \langle ?, ?, ?, ? \rangle$.
- **Sample 10 (Yes: Senior, Medium, Yes, Fair):** $\implies S_6 = \langle ?, ?, ?, ? \rangle$.

iii. **Final Learned Hypothesis:** $S_{\text{final}} = \langle ?, ?, ?, ? \rangle$. The learned hypothesis classifies all future instances as positive because the training positives encompass all possible attribute values.

Ramaiah Internal Assessment - II (May 2026) · Q 2(c) [6 Marks · Blooms L3, CO5]

"Using the covariance matrix Apply PCA: $C = \begin{bmatrix} 4 & 2 \\ 2 & 4 \end{bmatrix}$. 1. Find the Eigenvalues. 2. Find the Eigenvectors. 3. Identify the First Principal Component (PC1)."

1. Characteristic Equation & Eigenvalues:

$$\det(C - \lambda I) = \begin{vmatrix} 4 - \lambda & 2 \\ 2 & 4 - \lambda \end{vmatrix} = (4 - \lambda)^2 - (2)(2) = 0$$

$$\lambda^2 - 8\lambda + 16 - 4 = \lambda^2 - 8\lambda + 12 = 0$$

$$(\lambda - 6)(\lambda - 2) = 0 \implies \lambda_1 = \mathbf{6.0}, \quad \lambda_2 = \mathbf{2.0}$$

2. Eigenvectors:

• For $\lambda_1 = 6$: $(C - 6I)\mathbf{v} = \begin{bmatrix} -2 & 2 \\ 2 & -2 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \implies -2v_1 + 2v_2 = 0 \implies v_1 = v_2.$

Normalized Eigenvector: $\mathbf{e}_1 = \frac{1}{\sqrt{1^2+1^2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix} \approx \begin{bmatrix} 0.7071 \\ 0.7071 \end{bmatrix}.$

• For $\lambda_2 = 2$: $(C - 2I)\mathbf{v} = \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \implies 2v_1 + 2v_2 = 0 \implies v_1 = -v_2.$

Normalized Eigenvector: $\mathbf{e}_2 = \frac{1}{\sqrt{1^2+(-1)^2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{bmatrix} \approx \begin{bmatrix} 0.7071 \\ -0.7071 \end{bmatrix}.$

3. First Principal Component (PC1):

The First Principal Component corresponds to the largest eigenvalue $\lambda_1 = 6$:

$$\text{PC1 Axis} = \mathbf{e}_1 = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}$$

Proportion of Variance Explained: $\frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{6}{6+2} = \frac{6}{8} = \mathbf{75.00\%}$. PC1 alone captures 75% of the total data spread.

Ramaiah Internal Assessment – I (March 2026) · Q 3(b) [6 Marks · Blooms L2, CO2]

"Explain the Candidate Elimination Algorithm in concept learning."

The **Candidate Elimination Algorithm** addresses the fatal limitation of Find-S (which cannot detect ambiguous versions or determine if training instances are consistent). It computes the **Version Space** ($VS_{H,D}$) by incrementally updating two boundary sets:

- **Initialization:**

$$G_0 \leftarrow \{\langle ?, ?, \dots, ? \rangle\}, \quad S_0 \leftarrow \{\langle \phi, \phi, \dots, \phi \rangle\}$$

- **For each Positive Training Instance** $d = (x, +)$:

1. Remove from G any hypothesis inconsistent with d .
2. For each hypothesis $s \in S$ not consistent with d :
 - Remove s from S .
 - Add to S all minimal generalizations h of s such that h is consistent with d , and some member of G is more general than h .
 - Remove from S any hypothesis that is more general than another hypothesis in S .

- **For each Negative Training Instance** $d = (x, -)$:

1. Remove from S any hypothesis inconsistent with d (i.e. that incorrectly accepts d).
2. For each hypothesis $g \in G$ not consistent with d (i.e. that accepts d):
 - Remove g from G .
 - Add to G all minimal specializations h of g such that h is consistent with d , and some member of S is more specific than h .
 - Remove from G any hypothesis that is less general than another hypothesis in G .

- **Convergence:** The boundaries converge to a single hypothesis if the concept is learnable and error-free.

Dimension	Data Matrix (Object-by-Attribute)	Dissimilarity Matrix (Object-by-Object)
Structure	$n \times p$ matrix: n objects across p distinct attributes (two-mode).	$n \times n$ square symmetric matrix: pairwise distances between objects (one-mode).
Mathematical Form	$\mathbf{X} = [x_{ij}]_{n \times p}$ where x_{ij} is the value of attribute j for object i .	$\mathbf{D} = [d(i, j)]_{n \times n}$ where $d(i, j)$ is the distance metric between objects i and j .

Dimension	Data Matrix (Object-by-Attribute)	Dissimilarity Matrix (Object-by-Object)
Diagonal Elements	Arbitrary attribute measurements (no diagonal restriction).	Always strictly zero: $d(i, i) = 0$ (identity of indiscernibles).
Symmetry	Non-symmetric.	Symmetric: $d(i, j) = d(j, i)$.
Concrete Example	Dataset of 3 patients with Age and BMI: $\mathbf{X} = \begin{bmatrix} 25 & 22.4 \\ 45 & 28.1 \\ 60 & 31.0 \end{bmatrix}_{3 \times 2}$	Euclidean distance between the 3 patients: $\mathbf{D} = \begin{bmatrix} 0 & 20.8 & 36.0 \\ 20.8 & 0 & 15.3 \\ 36.0 & 15.3 & 0 \end{bmatrix}_{3 \times 3}$

Ramaiah SEE Examination (June/July 2026) · Q 3(c) [6 Marks · Blooms L2, CO2]

"Consider the dataset: Student (S1-S6), Study (High, Medium, Low), Attendance (Good, Poor), Assignment (Yes, No), Result (Pass/Fail). Frame the concept learning task and identify Instance space, Hypothesis space and Target concept."

- **Target Concept (c):** A boolean function $c : \mathcal{X} \rightarrow \{0, 1\}$ predicting whether a student will Pass:

$$c(x) = \begin{cases} 1(\text{Pass}) & \text{if student qualifies} \\ 0(\text{Fail}) & \text{otherwise} \end{cases}$$

- **Instance Space ($\mathcal{X}$):** The Cartesian product of all possible attribute values:

- Study $\in \{\text{High, Medium, Low}\}$ (3 values)
- Attendance $\in \{\text{Good, Poor}\}$ (2 values)
- Assignment $\in \{\text{Yes, No}\}$ (2 values)

Total distinct instances: $|\mathcal{X}| = 3 \times 2 \times 2 = \mathbf{12}$ instances.

- **Hypothesis Space ($\mathcal{H}$):** Conjunction of constraints on the 3 attributes. Each attribute slot can take a specific value, '?', or ' ϕ ':

- Number of syntactically distinct conjunctions: $(3 + 2) \times (2 + 2) \times (2 + 2) = 5 \times 4 \times 4 = \mathbf{80}$ hypotheses.

- Semantically distinct hypotheses (since any hypothesis containing ' ϕ ' represents the empty concept $\emptyset$):

$$|\mathcal{H}_{\text{sem}}| = (3 + 1) \times (2 + 1) \times (2 + 1) + 1 = 4 \times 3 \times 3 + 1 = \mathbf{37} \text{ distinct hypotheses}$$

Ramaiah SEE Examination (June/July 2026) · Q 4(c) [8 Marks · Blooms L2, CO2]

"Explain the working of Linear Discriminant Analysis (LDA) with suitable mathematical expressions."

Objective: Unlike PCA (which maximizes overall variance without regard to labels), **Linear Discriminant Analysis (LDA)** is a supervised dimensionality reduction technique that finds an optimal projection vector $\mathbf{w}$ maximizing class separability while minimizing intra-class dispersion.

- **1. Between-Class Scatter Matrix (S_B):** Measures the distance between class mean vectors $\mathbf{m}_1$ and $\mathbf{m}_2$:

$$S_B = (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^T$$

- **2. Within-Class Scatter Matrix (S_W):** Measures the internal spread (variance) of each class:

$$S_W = \sum_{\mathbf{x} \in C_1} (\mathbf{x} - \mathbf{m}_1)(\mathbf{x} - \mathbf{m}_1)^T + \sum_{\mathbf{x} \in C_2} (\mathbf{x} - \mathbf{m}_2)(\mathbf{x} - \mathbf{m}_2)^T$$

- **3. Fisher's Linear Discriminant Criterion:** Maximize the ratio of between-class variance to within-class variance:

$$J(\mathbf{w}) = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}$$

- **4. Optimal Solution:** Taking the derivative with respect to $\mathbf{w}$ and setting to zero yields the generalized eigenvalue problem $S_W^{-1} S_B \mathbf{w} = \lambda \mathbf{w}$. For two classes, since $S_B \mathbf{w}$ is in the direction of $(\mathbf{m}_1 - \mathbf{m}_2)$, the optimal projection vector is:

$$\mathbf{w}^* = S_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2)$$

Practice Problems (Step-by-Step Verified Solutions)

The following 6 practice problems have been mathematically verified via `audit/verify/ml/verify_ml_u2.py`. Attempt each manually before expanding the solution blocks.

Problem 1 (2D PCA Manual Trace): Consider 4 data points $X = [[2, 4], [3, 6], [5, 8], [6, 10]]$. (i) Compute the centered matrix X_c . (ii) Compute the Sample Covariance Matrix Σ . (iii) Derive eigenvalues λ_1, λ_2 and the principal eigenvector $\mathbf{v}_1$.

Practice Problem:

View Verified Solution

Solution:

Mean vector: $\bar{x}_1 = \frac{2+3+5+6}{4} = 4.0$, $\bar{x}_2 = \frac{4+6+8+10}{4} = 7.0$.

- (i) **Centered Matrix X_c :**

$$X_c = \begin{bmatrix} 2-4 & 4-7 \\ 3-4 & 6-7 \\ 5-4 & 8-7 \\ 6-4 & 10-7 \end{bmatrix} = \begin{bmatrix} -2 & -3 \\ -1 & -1 \\ 1 & 1 \\ 2 & 3 \end{bmatrix}$$

- (ii) **Covariance Matrix ($n - 1 = 3$):**

$$\cdot \text{Var}(x_1) = \frac{(-2)^2 + (-1)^2 + 1^2 + 2^2}{3} = \frac{10}{3} \approx \mathbf{3.3333}$$

$$\cdot \text{Var}(x_2) = \frac{(-3)^2 + (-1)^2 + 1^2 + 3^2}{3} = \frac{20}{3} \approx \mathbf{6.6667}$$

$$\cdot \text{Cov}(x_1, x_2) = \frac{(-2)(-3) + (-1)(-1) + (1)(1) + (2)(3)}{3} = \frac{6+1+1+6}{3} = \frac{14}{3} \approx \mathbf{4.6667}$$

$$\Sigma = \begin{bmatrix} 3.3333 & 4.6667 \\ 4.6667 & 6.6667 \end{bmatrix}$$

- (iii) **Eigenvalues & PC1:**

$$\text{Trace} = 10.0, \quad \text{Det} = \left(\frac{10}{3}\right) \left(\frac{20}{3}\right) - \left(\frac{14}{3}\right)^2 = \frac{200 - 196}{9}$$

$$\lambda^2 - 10\lambda + \frac{4}{9} = 0 \implies \lambda_1 \approx \mathbf{9.9554}, \quad \lambda_2 \approx \mathbf{0.0446}$$

$$\mathbf{v}_1 \approx [0.5760, 0.8174]^T \implies \text{Var Explained} = \frac{\mathbf{9.9554}}{\mathbf{10.0}} = \mathbf{99.55\%}$$

Problem 2 (Find-S Algorithm Trace): Trace Find-S on Mitchell's 4 training examples:

D1: (Sunny, Warm, Normal, Strong, Warm, Same) $\rightarrow$ Yes

D2: (Sunny, Warm, High, Strong, Warm, Same) $\rightarrow$ Yes

D3: (Rainy, Cold, High, Strong, Warm, Change) $\rightarrow$ No

D4: (Sunny, Warm, High, Strong, Cool, Change) $\rightarrow$ Yes

Practice Problem:

View Verified Solution

Solution:

- $h_0 = \langle \phi, \phi, \phi, \phi, \phi, \phi \rangle$
- D_1 (Yes) $\implies h_1 = \langle \text{Sunny, Warm, Normal, Strong, Warm, Same} \rangle$
- D_2 (Yes) $\implies h_2 = \langle \text{Sunny, Warm, ?, Strong, Warm, Same} \rangle$
- D_3 (No) $\implies h_3 = h_2$ (Negative instance ignored by Find-S)
- D_4 (Yes) $\implies h_4 = \langle \text{Sunny, Warm, ?, Strong, ?, ?} \rangle$

Problem 3 (Candidate Elimination Trace): Apply Candidate Elimination to the 4 EnjoySport instances above. Show S_i and G_i at each step.

Practice Problem:

View Verified Solution

Solution:

- $S_0 = \{\langle \phi, \phi, \phi, \phi, \phi, \phi \rangle\}$, $G_0 = \{\langle ?, ?, ?, ?, ?, ? \rangle\}$
- $D_1(+) \implies S_1 = \{\langle \text{Sunny}, \text{Warm}, \text{Normal}, \text{Strong}, \text{Warm}, \text{Same} \rangle\}$, $G_1 = G_0$
- $D_2(+) \implies S_2 = \{\langle \text{Sunny}, \text{Warm}, ?, \text{Strong}, \text{Warm}, \text{Same} \rangle\}$, $G_2 = G_0$
- $D_3(-) \implies S_3 = S_2$, $G_3 = \{\langle \text{Sunny}, ?, ?, ?, ?, ? \rangle, \langle ?, \text{Warm}, ?, ?, ?, ? \rangle, \langle ?, ?, ?, ?, ?, \text{Same} \rangle\}$
- $D_4(+) \implies S_4 = \{\langle \text{Sunny}, \text{Warm}, ?, \text{Strong}, ?, ? \rangle\}$, $G_4 = \{\langle \text{Sunny}, ?, ?, ?, ?, ? \rangle, \langle ?, \text{Warm}, ?, ?, ?, ? \rangle\}$

Problem 4 (PAC Learnability Sample Bound): A hypothesis space contains $|\mathcal{H}| = 973$ conjunctions. How many training examples m are required to guarantee with confidence 95% ($\delta = 0.05$) that the learned hypothesis has true error $\leq 10\%$ ($\epsilon = 0.1$)?

Practice Problem:

[View Verified Solution](#)

Solution:

Under Valiant's Probably Approximately Correct (PAC) framework:

$$m \geq \frac{1}{\epsilon} \left(\ln |\mathcal{H}| + \ln \frac{1}{\delta} \right)$$

- $\ln(973) \approx 6.8804$
- $\ln\left(\frac{1}{0.05}\right) = \ln(20) \approx 2.9957$
- $m \geq \frac{1}{0.1} (6.8804 + 2.9957) = 10 \times 9.8761 = \mathbf{98.761}$
- **Conclusion:** At least $m = \mathbf{99}$ training samples are mathematically guaranteed to ensure PAC learnability.

Problem 5 (Ordinal Dissimilarity Computation): Three candidates obtain grade ratings in an interview: Candidate A: 'Good', Candidate B: 'Poor', Candidate C: 'Average'. The scale is Poor(1) < Average(2) < Good(3). Compute the pairwise normalized dissimilarity matrix.

Practice Problem:

[View Verified Solution](#)

Solution:

$$M_f = 3, \quad z_{if} = \frac{r_{if}-1}{3-1} = \frac{r_{if}-1}{2}.$$

- $z_A = \frac{3-1}{2} = 1.00$
- $z_B = \frac{1-1}{2} = 0.00$
- $z_C = \frac{2-1}{2} = 0.50$
- Dissimilarity Matrix:

$$\mathbf{D} = \begin{bmatrix} 0 & 1.00 & 0.50 \\ 1.00 & 0 & 0.50 \\ 0.50 & 0.50 & 0 \end{bmatrix}$$

Problem 6 (Design of a Learning System): Outline Tom Mitchell's four design choices for developing an AI Autonomous Driving Lane Assistant.

Practice Problem:

[View Verified Solution](#)

Solution:

1. **1. Choosing the Training Experience:** Streaming dashcam RGB video sequences paired with human telemetry (steering angle in radians, throttle percentage, braking force).
2. **2. Choosing the Target Function:** $V : \text{Image Frame } \mathbf{X} \rightarrow \text{Steering Angle } \hat{\theta} \in [-1.0, +1.0]$.
3. **3. Choosing Representation:** A deep convolutional neural network (CNN) parameterized by weight tensors $\mathbf{W}$ mapping visual feature maps to scalar control.
4. **4. Choosing Approximation Algorithm:** Stochastic Gradient Descent (Adam optimizer) minimizing Mean Squared Error loss $\mathcal{L} = \frac{1}{N} \sum (\theta_i - \hat{\theta}_i)^2$ with L_2 weight decay.